

Universidad de Costa Rica

Facultad de Ingeniería

Escuela de Ciencias de la Computación e Informática

Bases de Datos Avanzadas

Laboratorio de Neo4j



Profesor

Luis Gustavo Esquivel

Estudiante

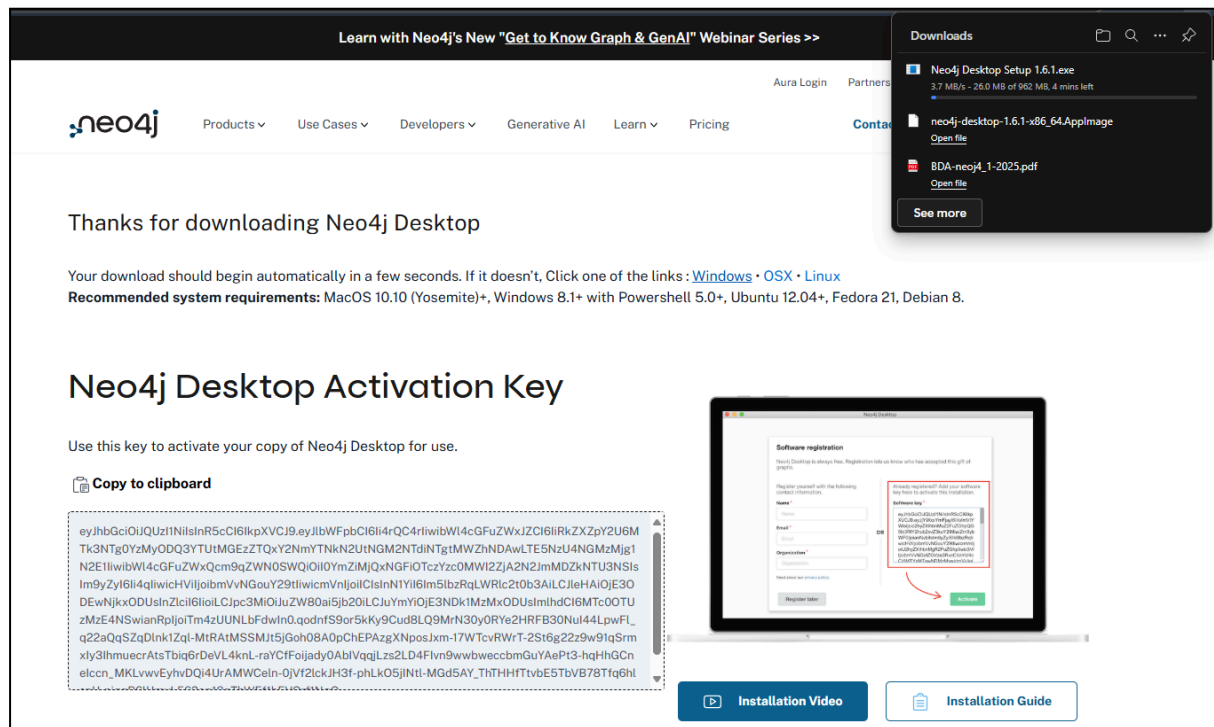
Brandon Trigueros Lara C17899

Semestre I

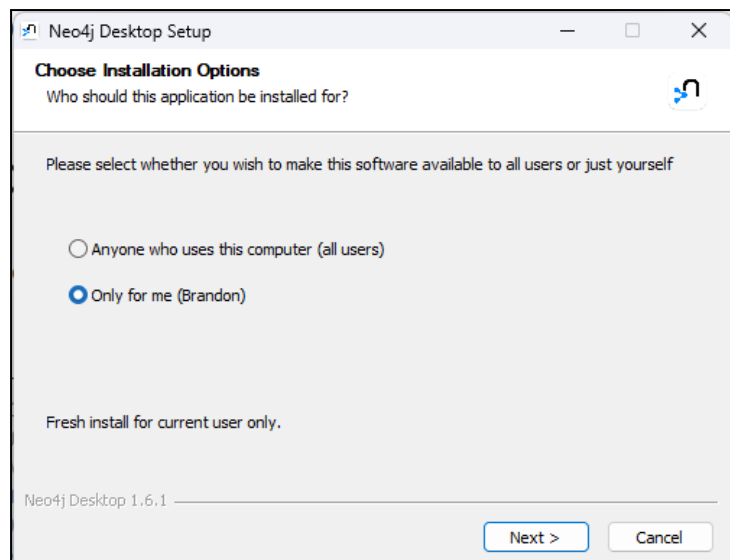
2025

1.

a)



b) Se abre el ejecutable descargado y se selecciona instalar solo para el usuario actual y se clicka siguiente.



Se acepta la ubicación de instalación por defecto y clic a instalar.

c) Con Neo4j instalado se procede a abrirlo y a digitar las credenciales para activarlo.

Software registration

Activation is optional. Neo4j Desktop is free for local development use.

Activation helps us improve our products. Registration identifies you for commercial agreements and support. You can also find support in our [community forums](#) and our [developer center](#).

Register yourself with the following contact information.

Name *

Email *

Organization *

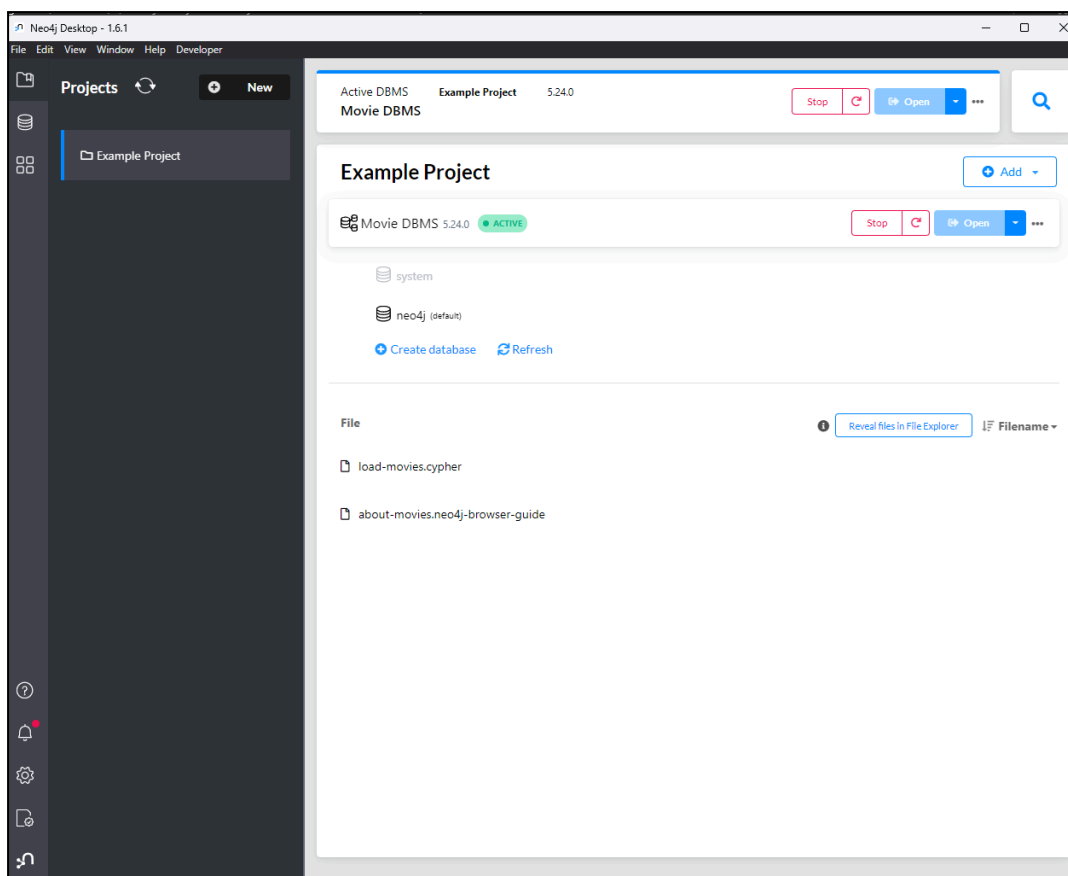
[Read about our privacy policy.](#)

OR

Software key *

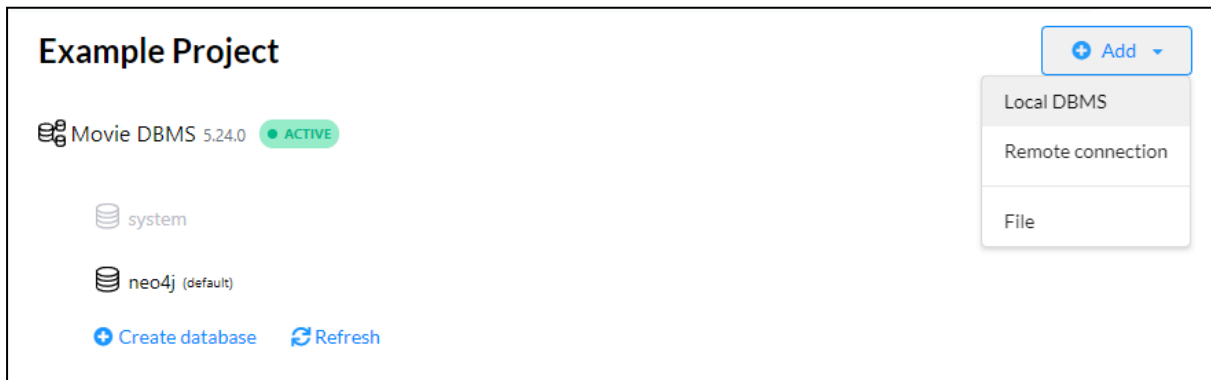
Software keys look like a long block of hexadecimal characters.

Neo4j terminará de configurarse y aparece su ventana de inicio

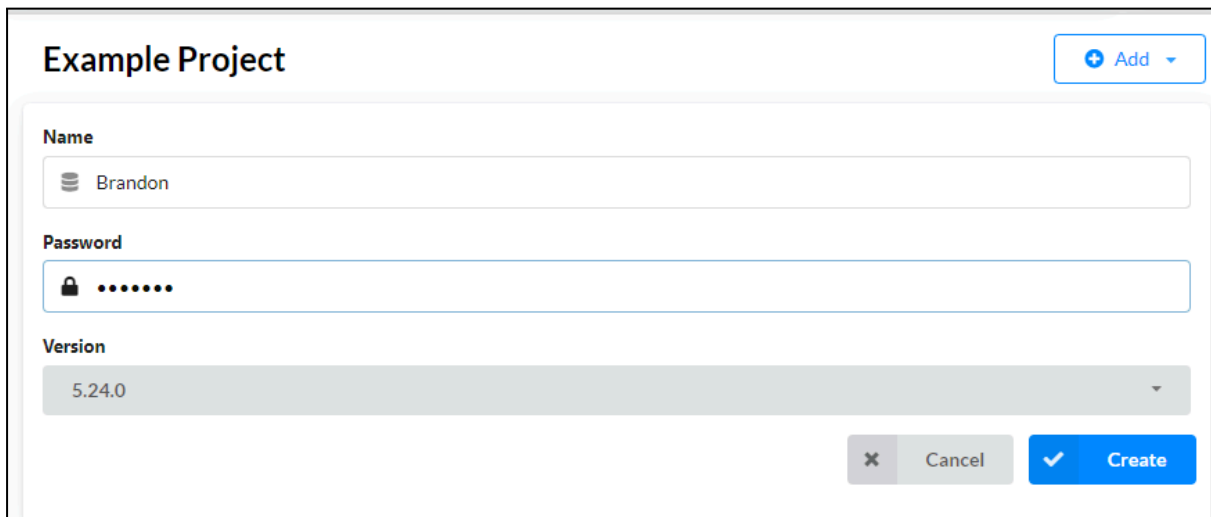


2. Con Neo4j abierto, se realiza lo siguiente

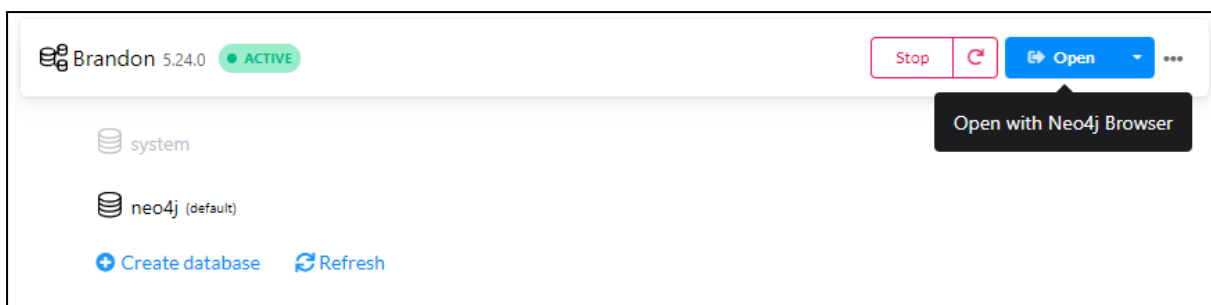
a) En el primer proyecto incluido por defecto se hace clic en Add y luego Local DBMS

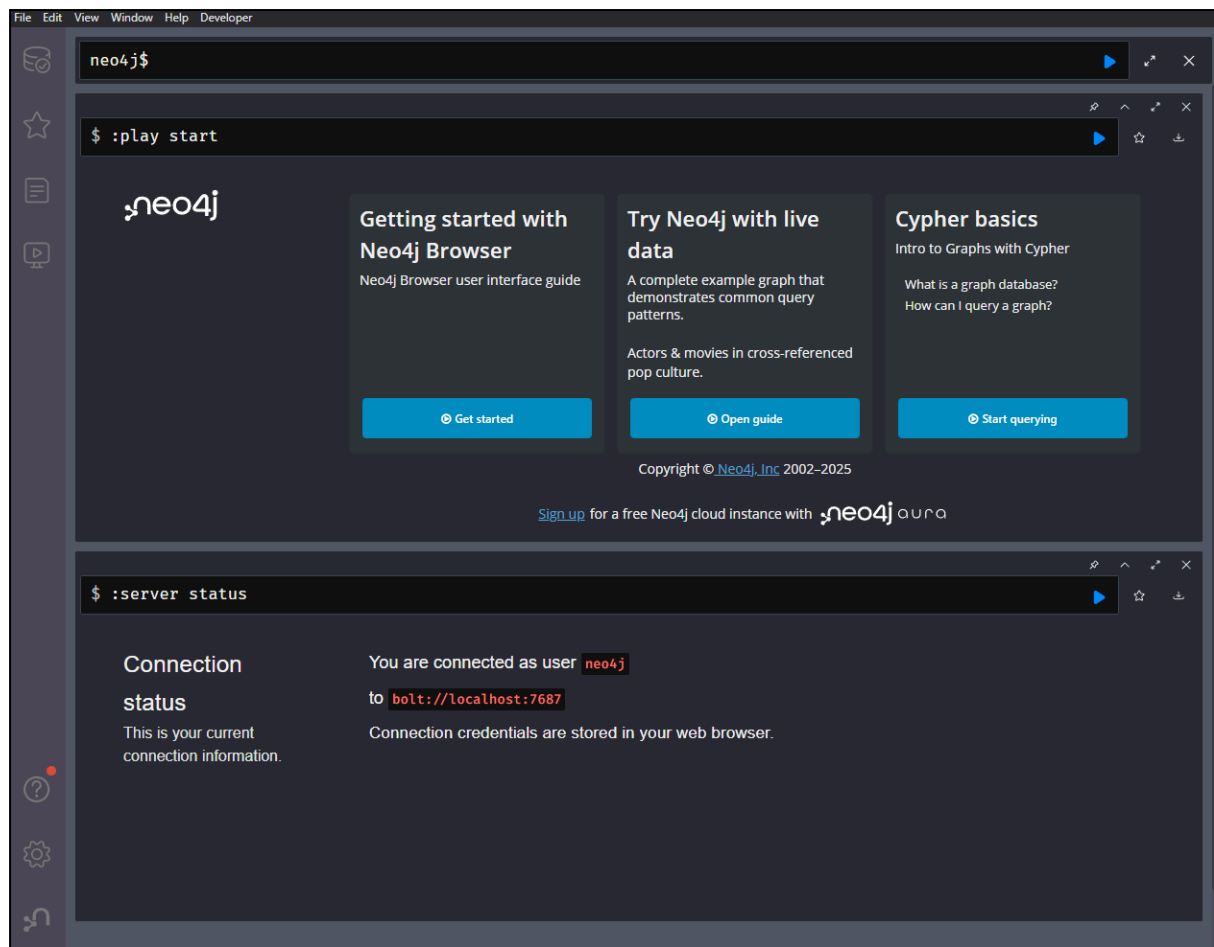


Se nombra la base de datos y se asigna una contraseña

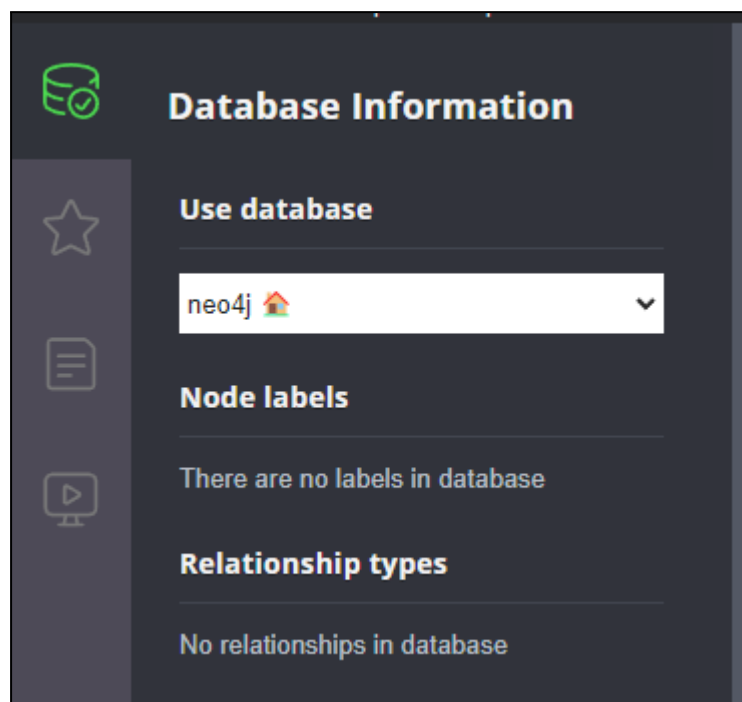


b) Se inicia la nueva base de datos y se abre ('Open') en Neo4j Browser.





c) En el panel izquierdo se selecciona Database para ver lo siguiente



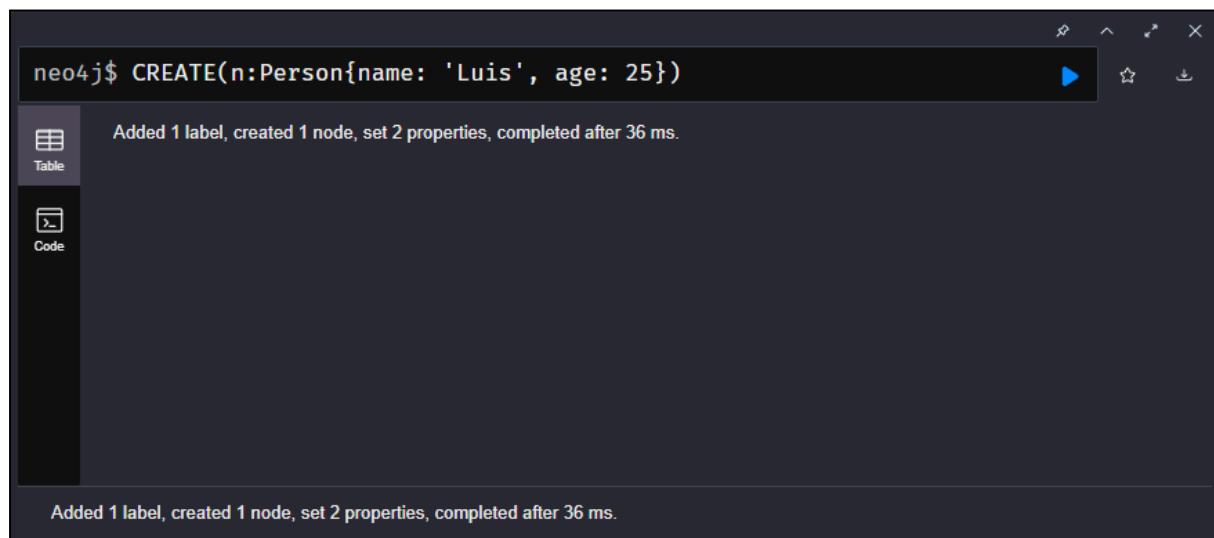
Vemos que la base de datos no tiene actualmente etiquetas o relaciones.

relaciones existen en esta base de datos.

d) Se ejecuta la siguiente consulta: `CREATE (n:Person{name: 'Luis', age: 25})`

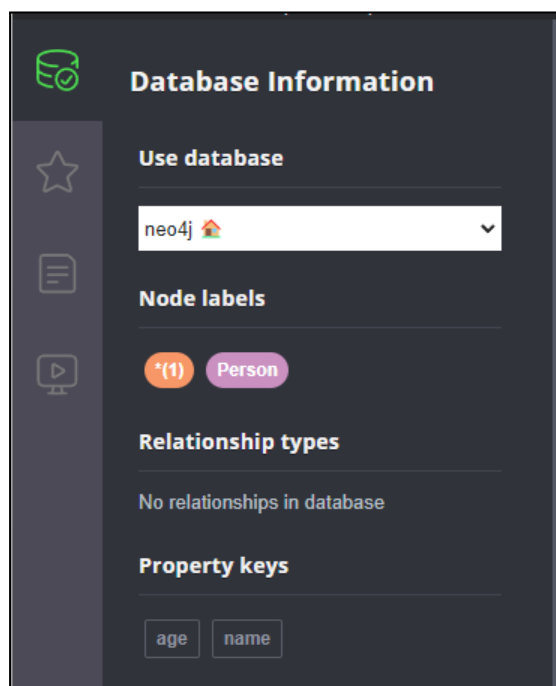
```
neo4j$ CREATE(n:Person{name: 'Luis', age: 25})
```

Se obtiene el resultado



El cual está indicando que se añadió una etiqueta, que se creó un nodo y que se establecieron 2 propiedades en 36 milisegundos.

e) También se puede apreciar en el panel Database de la izquierda ahora tenemos un Nodo etiqueta y dos propiedades clave (age y name).



f) Si hacemos click sobre la etiqueta Persona podemos ver el nodo de etiqueta de manera gráfica



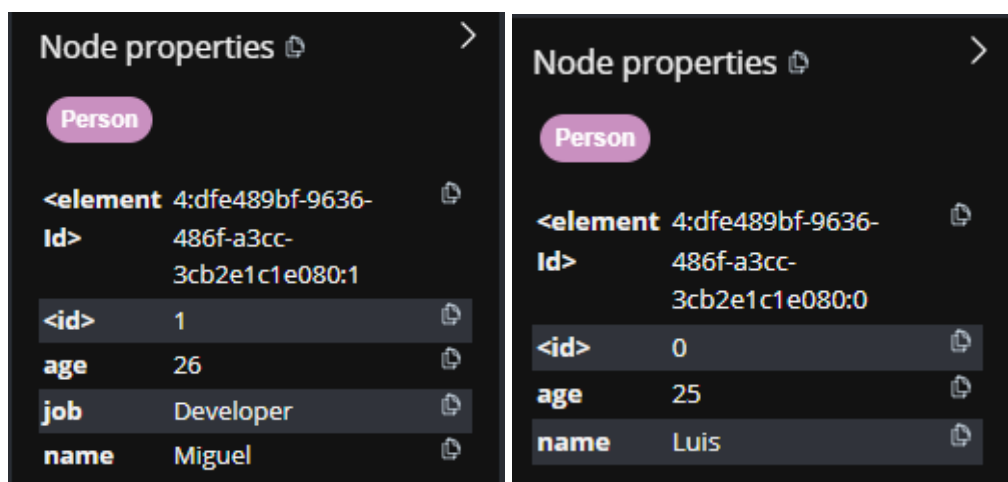
g) Se ejecuta la siguiente consulta: `CREATE (n:Person{name: 'Miguel', age: 26, job: 'Developer'})`



h) Se revisa nuevamente la etiqueta Person para ver sus nodos. El resultado que obtenemos es el siguiente.



Además podemos ver los atributos de cada uno dando click sobre cada nodo.



Es apreciable que ambos nodos comparten que son de la etiqueta Person, ambos tienen en común que cuentan con los atributos `<id>`, `age` y `name`. Se diferencian en que el nodo Miguel tiene además una característica llamada `job` la cual Luis no tiene aunque ambos son de la etiqueta Person. En SQL, el esquema es fijo y no permite fácilmente filas con columnas distintas como sí se permite aquí con Neo4j.

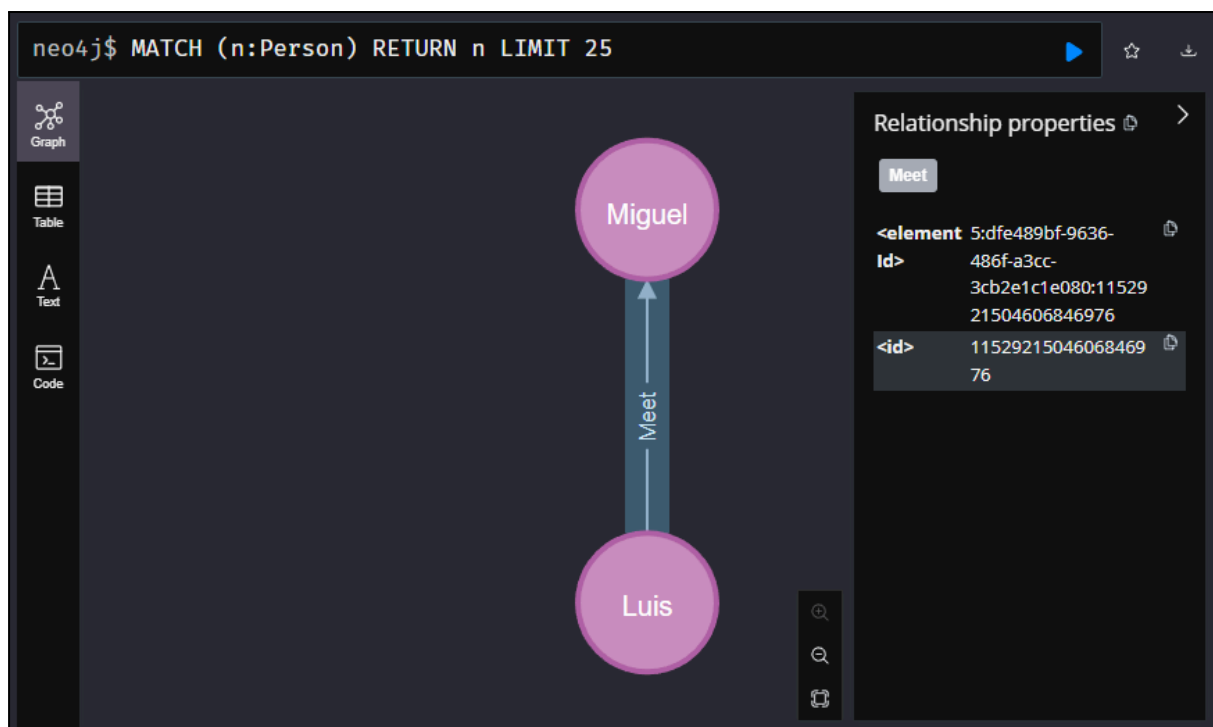
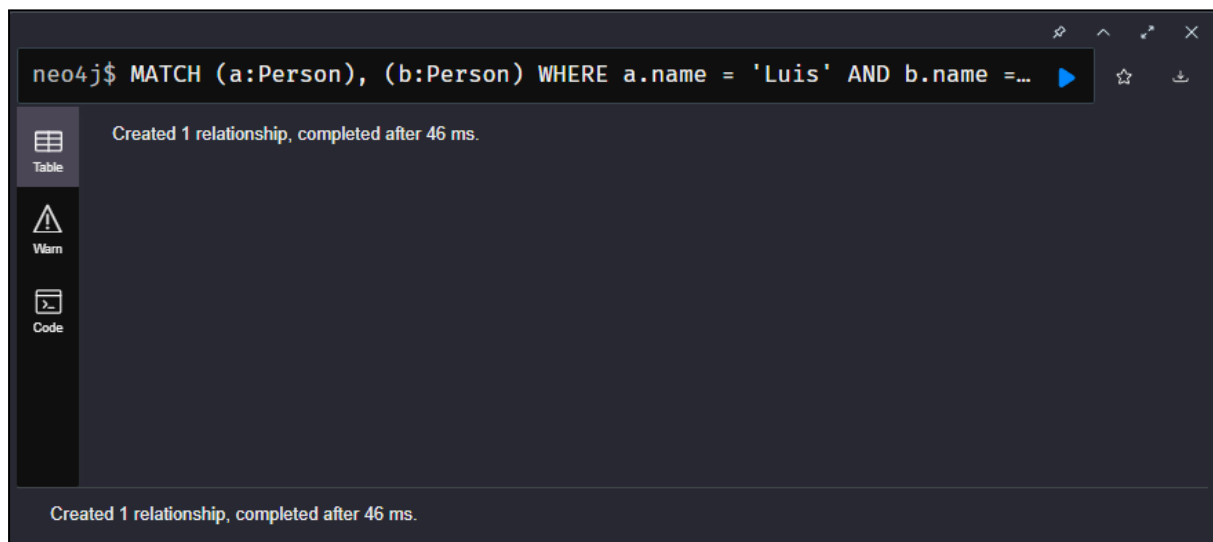
i) Se ejecuta la siguiente consulta:

```
MATCH (a:Person), (b:Person)
```

```
WHERE a.name = 'Luis' AND b.name = 'Miguel'
```

```
CREATE (a)-[r:Meet]->(b)
```


Se obtiene el siguiente resultado



En esta ocasión se nos indica que se ha creado una relación. Al consultar a Person vemos que se creó la relación Luis meet Miguel como se indicó en la consulta.

j) Ahora se ejecuta la siguiente consulta:

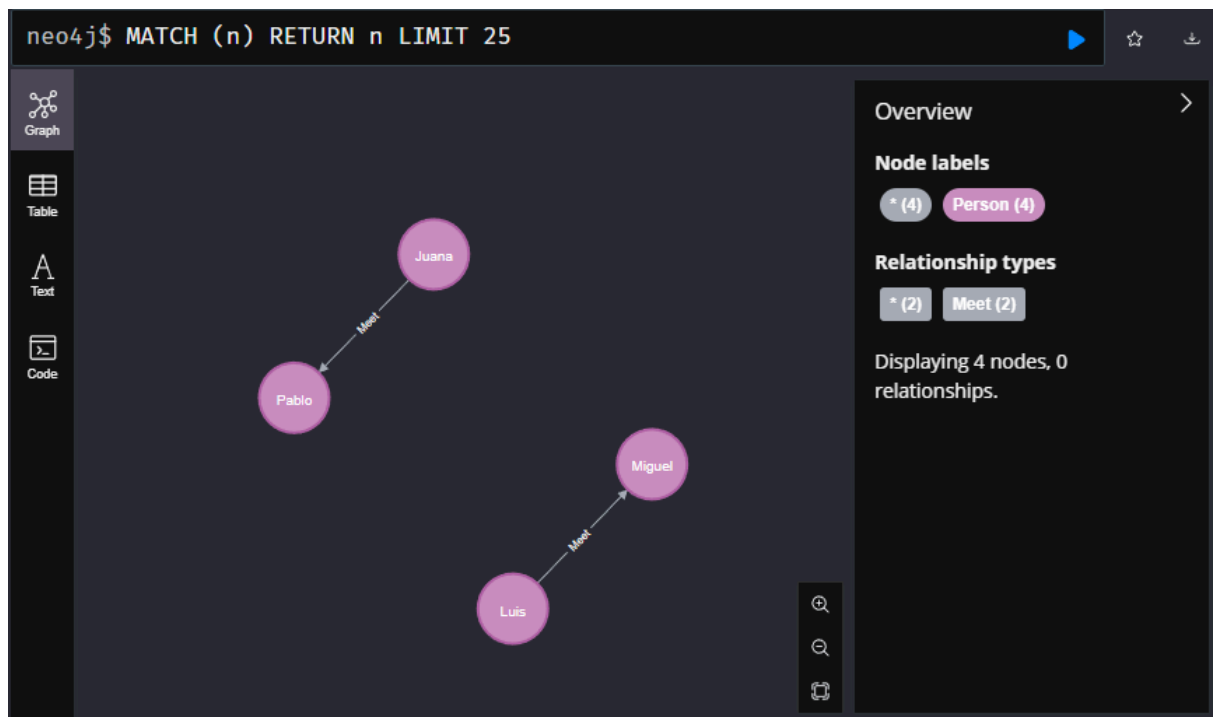
```
CREATE (:Person{name: 'Juana'})-[:Meet {desde: 2021}]-> (:Person{name: 'Pablo'})
```

```
neo4j$ CREATE (:Person{name: 'Juana'})-[:Meet {desde: 2021}]-> (:Pers...
```

Added 2 labels, created 2 nodes, set 3 properties, created 1 relationship, completed after 24 ms.

El resultado de la consulta dice que se crearon 2 etiquetas, 2 nodos y se asignaron 3 propiedades. En este caso los nodos etiqueta que se crearon fueron Juana y Pablo, la relación fue Meet y las propiedades fueron ambos nombres y la propiedad de relación 'desde'.

k) La relación Luis→Miguel no tiene propiedades, mientras que Juana→Pablo incluye la propiedad desde con el valor 2021.



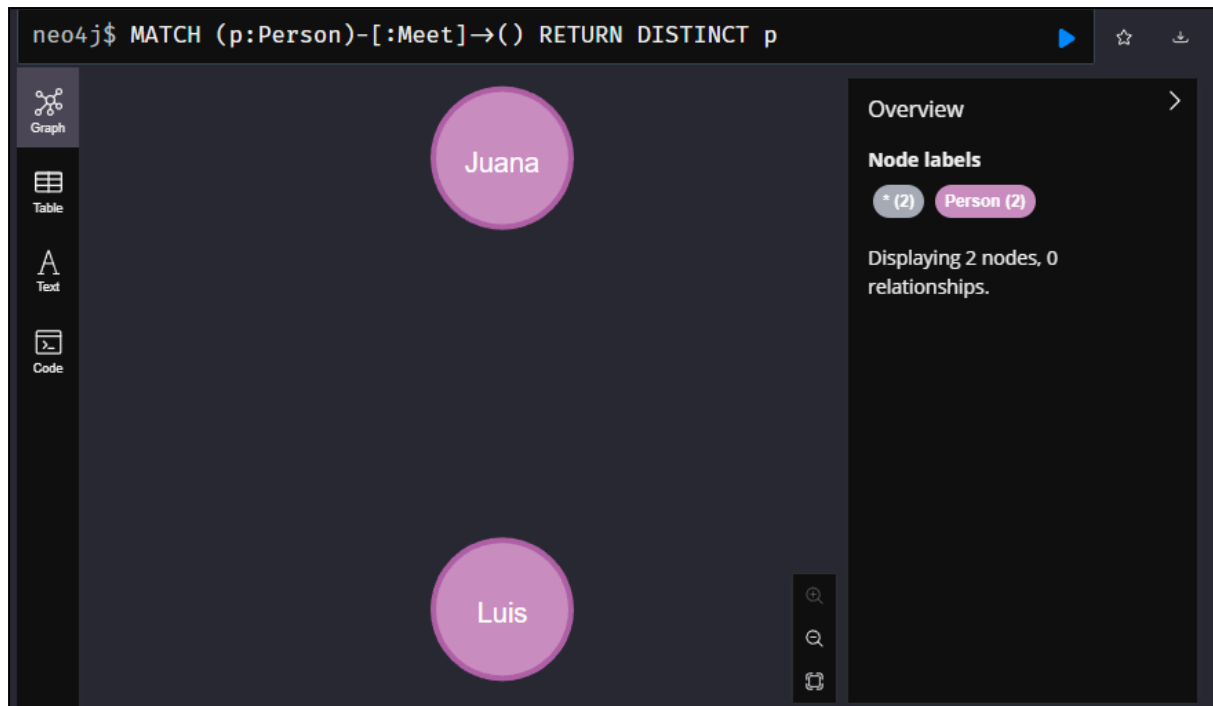
l) Neo4j es un grafo de propiedades dirigido etiquetado (property graph): esto quiere decir que nodos y relaciones llevan pares clave-valor.

m) Se ejecuta la siguiente consulta

```
MATCH (p:Person)-[:Meet]->()
```

```
RETURN DISTINCT p
```

Obtenemos el resultado

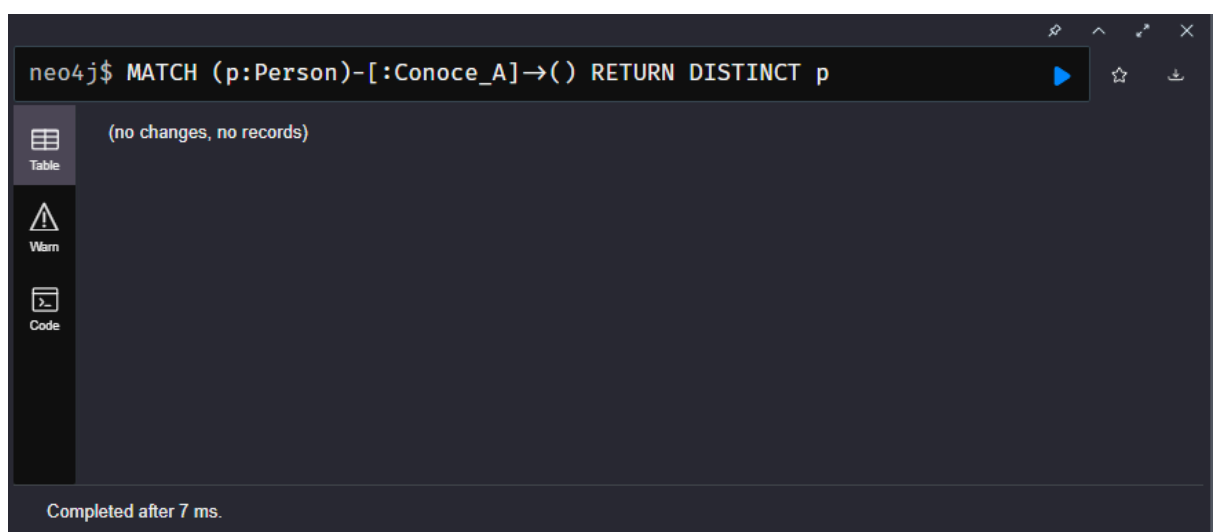


Se retornan solamente dos nodos como resultado porque la consulta lo que pide es las Personas con relación Meet salientes.

n) Se modifica la consulta a:

```
MATCH (p:Person)-[:Conoce_A]->()
```

```
RETURN DISTINCT p
```



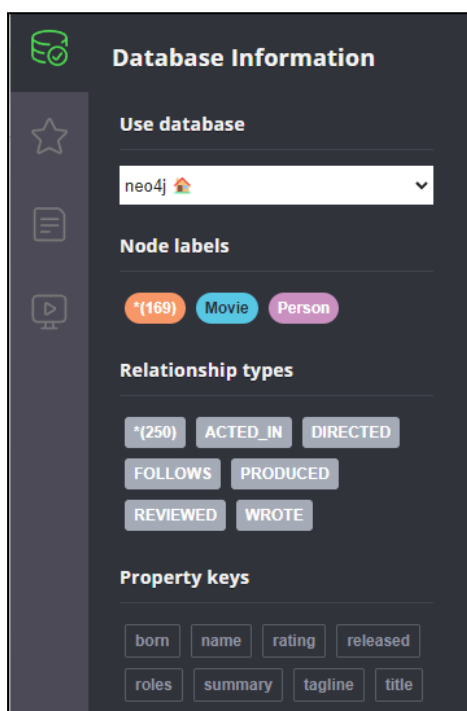
ñ) Se cierra Neo4j Browser y se detiene esta base de datos.



3. Al instalar Neo4j, este incluye por defecto una base de datos llamada Movie Database.

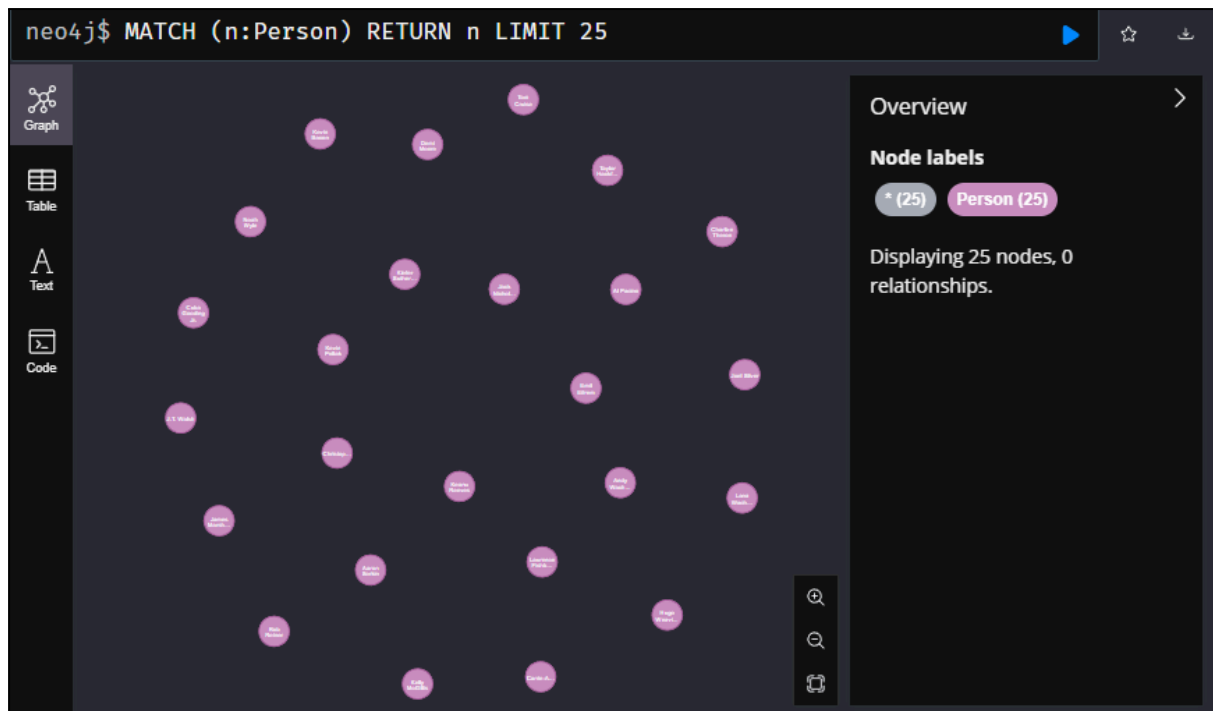
Se procede a abrir esta base de datos en Neo4j Browser y a realizar los siguientes puntos:

a)



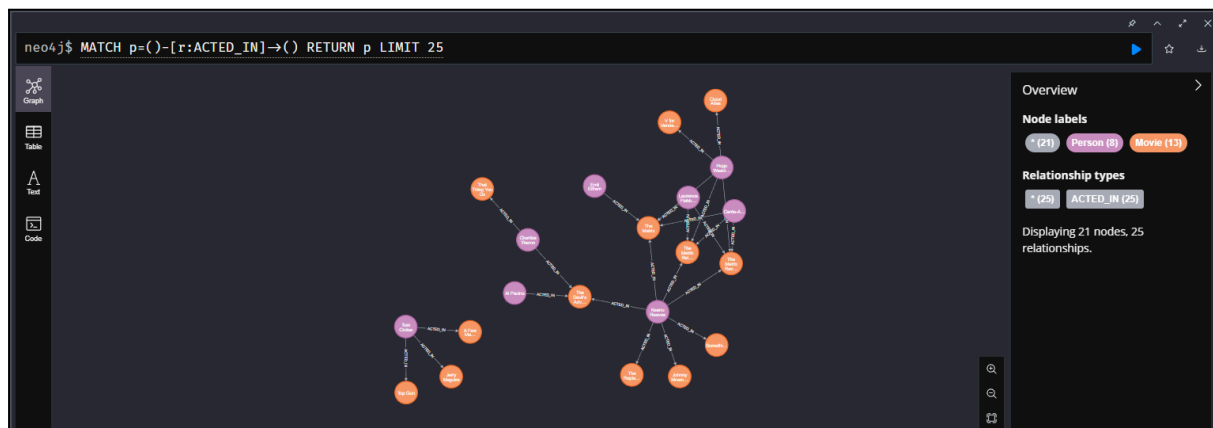
Los tipos de nodos presentes son Movie y Person. Las relaciones observables son ACTED_IN, DIRECTED, FOLLOWS, PRODUCED, REVIEWED y WROTE.

b) Al hacer click sobre el tipo de nodo Person



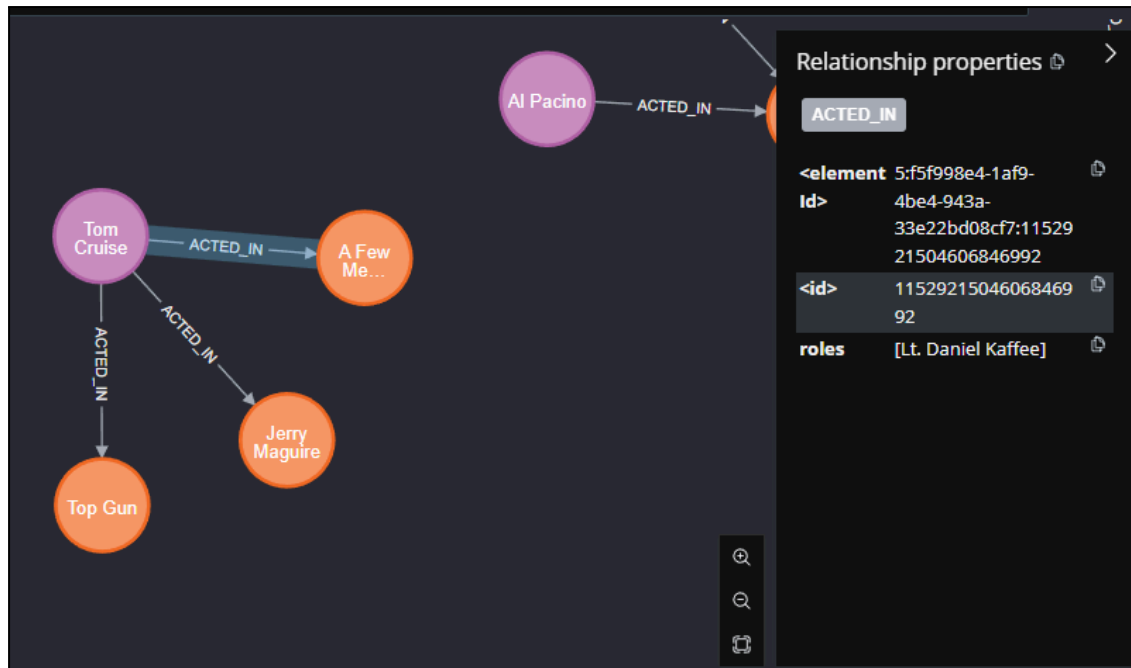
Parece ejecutar una consulta de MATCH, la cual devuelve 25 nodos del tipo Person, estos no tienen relaciones entre sí y tienen como propiedades name y born.

c) Al hacer click sobre la relación ACTED_IN.



El resultado me muestra todas las relaciones ACTED_IN, con lo cual podemos saber quien actuó en qué película. En estas relaciones interactúan dos tipos diferentes de nodos.

d)



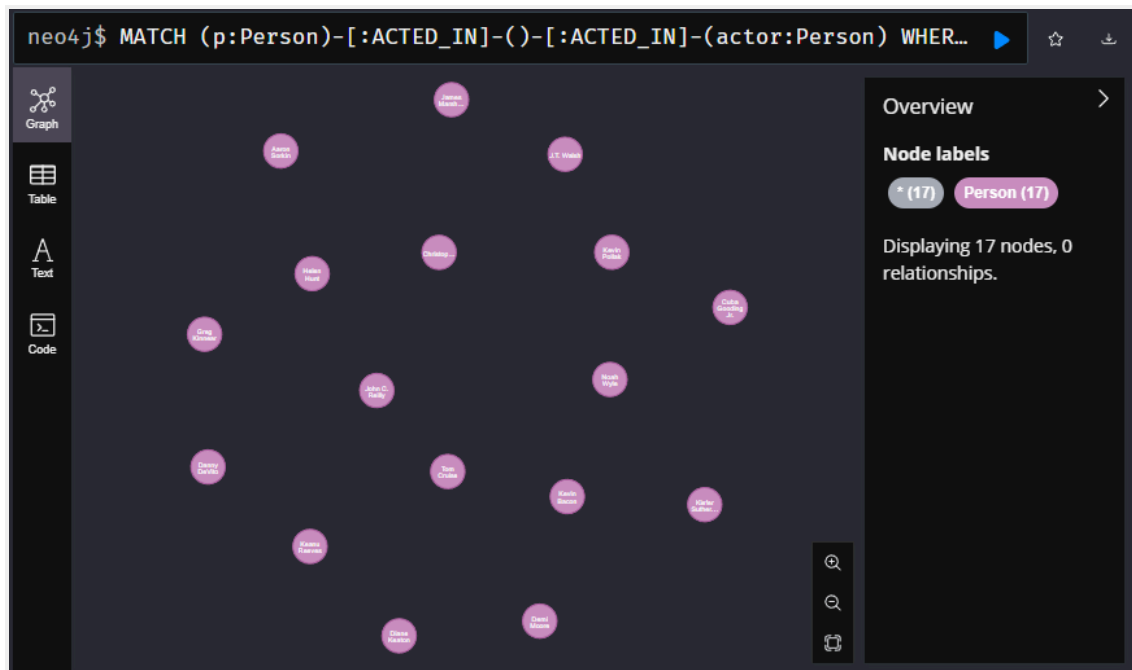
En los atributos de la relación `ACTED_IN` entre Tom Cruise y la película A Few Good Men se puede observar que el actor tuvo el rol de Lt. Daniel Kaffee.

e) Se ejecuta la siguiente consulta:

```
MATCH (p:Person)-[:ACTED_IN]-()-[:ACTED_IN]-(actor:Person)
```

```
WHERE p.name = "Jack Nicholson"
```

```
RETURN DISTINCT actor
```

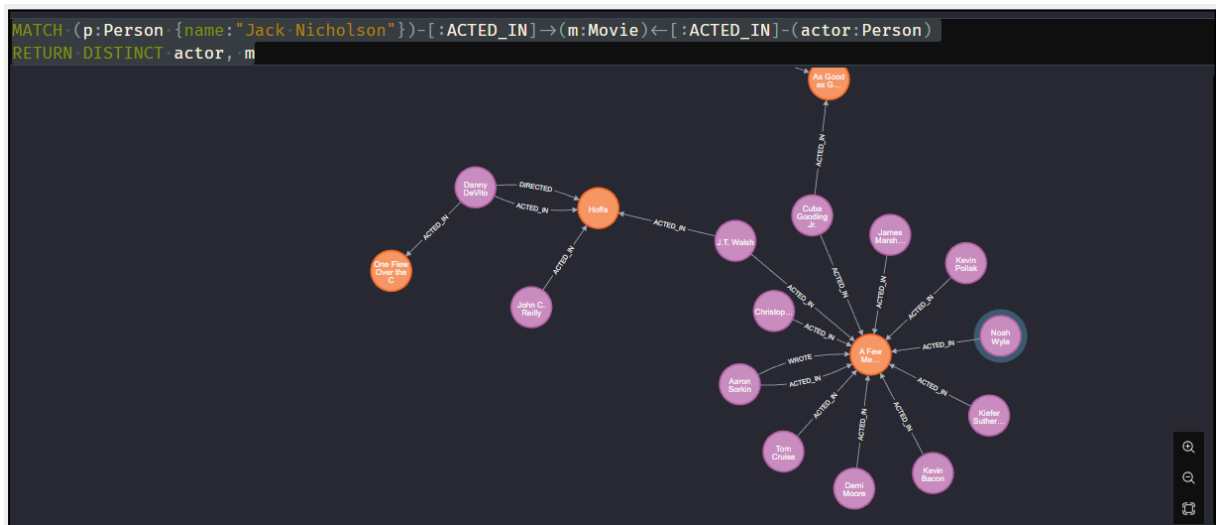


La consulta me devuelve todos aquellos actores que actuaron en una película junto con el actor p, donde p, en este caso, es Jack Nicholson. Devuelve actores como **Danny DeVito** que coincidieron con Jack Nicholson en alguna película.

f)

```
MATCH(p:Person{name:"Jack Nicholson"})-[:ACTED_IN]->(m:Movie)-[:ACTED_IN]-(actor:Person)
```

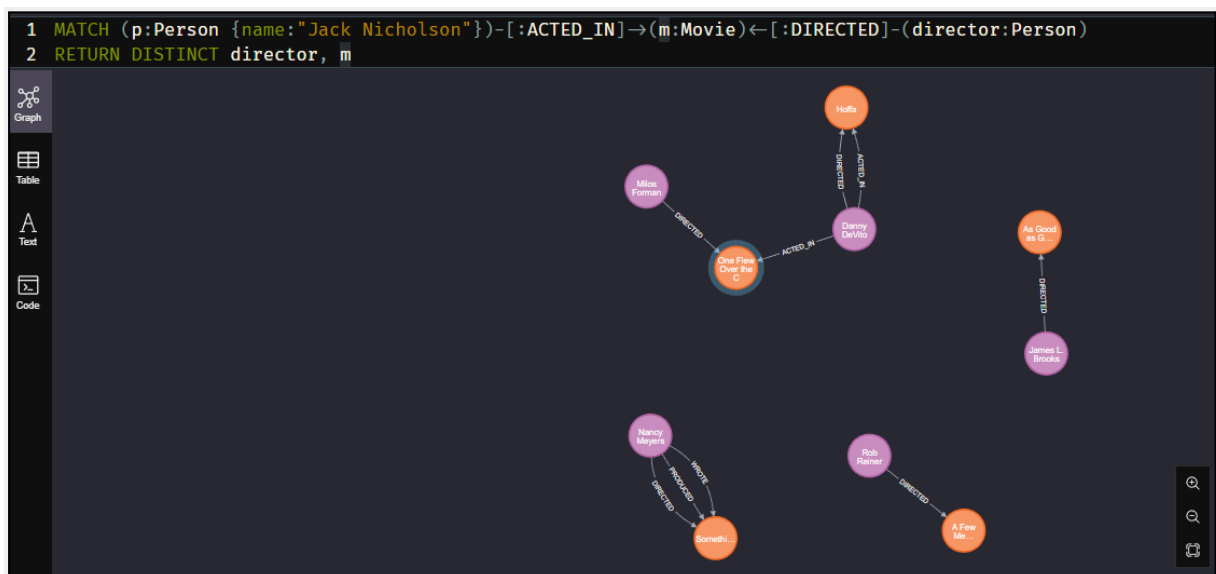
```
RETURN DISTINCT actor, m
```



g)

```
MATCH(p:Person{name:"Jack Nicholson"})-[:ACTED_IN]->(m:Movie)-[:DIRECTED]-(director:Person)
```

```
RETURN DISTINCT director, m
```



h)

MATCH

```
(jack:Person {name: "Jack  
Nicholson"})-[:ACTED_IN]->(:Movie)<-[:ACTED_IN]-(actor:Person)-[:ACTED_IN]->(pelicula:Movie)
```

WHERE

```
NOT (jack)-[:ACTED_IN]->(pelicula)
```

RETURN

```
DISTINCT actor.name AS Actor,
```

```
pelicula.title AS Película
```

```
neo4j$ MATCH (jack:Person {name: "Jack Nicholson"})-[:ACTED_IN]->(:Movie)<-[:ACTED_IN]-(actor:Person)-[:ACTED_IN]->(pelicula:Movie)
```

	Actor	Película
1	"Tom Cruise"	"Top Gun"
2	"Tom Cruise"	"Jerry Maguire"
3	"Kevin Bacon"	"Frost/Nixon"
4	"Kevin Bacon"	"Apollo 13"
5	"Kiefer Sutherland"	"Stand By Me"
6	"Cuba Gooding Jr."	"Jerry Maguire"
7	"Cuba Gooding Jr."	"What Dreams May Come"
8	"Helen Hunt"	"Twister"
9	"Helen Hunt"	"Cast Away"
10	"Greg Kinnear"	"You've Got Mail"
11	"Keanu Reeves"	"The Matrix"
12	"Keanu Reeves"	"The Matrix Reloaded"
13	"Keanu Reeves"	"The Matrix Revolutions"
14	"Keanu Reeves"	"The Devil's Advocate"
15	"Keanu Reeves"	"The Replacements"
16	"Keanu Reeves"	"Johnny Mnemonic"

i) Dos funciones que pueden tener consultas similares pueden ser las siguientes:

Recomendar amigos en redes sociales: buscás usuarios a los que aún no estás conectado pero que comparten amigos en común, ampliando tu red de manera natural.

Sugerir productos en e-commerce: encontrás artículos que compraron clientes con historiales de compra parecidos al tuyo, pero que vos aún no adquiriste, mejorando la personalización de ofertas.

4.

Detección de Redes de Fraude con neo4j:

Ambos videos muestran el poder de los grafos para exponer relaciones ocultas y acelerar la toma de decisiones en dos dominios clave. En “Detección de Redes de Fraude con Neo4j” se modela un dataset de transacciones como un grafo, aplicando algoritmos de centralidad y detección de comunidades para descubrir agrupaciones de usuarios que comparten recursos (tarjetas, dispositivos, IP) y añadir etiquetas de “riesgo”, casi duplicando la detección de fraude. En “Cyberseguridad con Neo4j” se construye un “gemelo digital” de toda la infraestructura —aplicaciones, balanceadores, servidores, racks, datacenters, alertas— permitiendo a los defensores explorar dependencias sin esquemas rígidos y ubicar al instante la causa raíz de incidentes, equilibrando así la ventaja frente a ataques cada vez más sofisticados.

Otras ideas futuras

Integración de grafos de confianza y reputación: combinar datos de reputación de usuarios, proveedores o dominios con grafos de transacciones para ajustar dinámicamente las puntuaciones de riesgo.

Gamificación de la seguridad: crear dashboards interactivos tipo “misiones” para equipos de seguridad, donde cada hallazgo en el grafo permite avanzar niveles y refinar reglas cooperativas.

Referencias

Neo4j, Inc., Neo4j Tutorial, 2022. dirección: <https://neo4j.com/developer/get-started/>

Neo4j, “Getting Started with Neo4j Desktop V1.2.3 on Linux,” YouTube, 12 Nov. 2019. Accessed: 10 June 2025. [Online Video]. Available: <https://www.youtube.com/watch?v=qyu1IHjJh-c>

P. Delano, “Detección de Redes de Fraude con Neo4j,” YouTube, Jun. 2024. Accessed: 10 June 2025. [Online Video]. Available: <https://www.youtube.com/watch?v=UwxT2m0kOxc>

P. Delano, “Cyberseguridad con Neo4j,” YouTube, Jun. 2023. Accessed: 10 June 2025. [Online Video]. Available: <https://www.youtube.com/watch?v=1fRm-b4dqe8>